

NAGARIKID

Digital Identity Platform

Master System Prompt & Build Specification

PERN Stack · AI Biometric Auth · Policy-Based Access · Full-Stack Implementation Guide

Stack	PostgreSQL · Express.js · React (Next.js) · Node.js
Authentication	JWT RS256 · bcrypt · Biometric face-match simulation
Deployment	Supabase · Render · Vercel (CI/CD on main branch)
Version	v2.0 — 2026
Document type	AI generation prompt + implementation spec

© 2026 NagarikID — Nepal Digital Identity Initiative. All rights reserved.

CONTENTS

Table of Contents

- 1. System Overview & Purpose3
- 2. Master AI Generation Prompt4
- 3. Database Architecture (PostgreSQL)7
- 4. Backend API Specification (Express + Node)10
- 5. Frontend Design System13
- 6. Authentication & Security Model15
- 7. AI & Biometric Integration17
- 8. Deployment Runbook18
- 9. Environment Variables Reference20
- 10. Implementation Checklist21

SECTION 1

System Overview & Purpose

NagarikID is Nepal's sovereign digital identity platform. It connects the government-issued National ID (NID) and Nagarik App ecosystem into a self-maintained, AI-augmented KYC and verification system. Citizens control what data gets shared, organizations get only what policy allows, and every access decision is permanently logged.

What the system solves

- Citizens currently present physical ID cards — prone to fraud, loss, and privacy leakage.
- Verifying organizations (banks, pharmacies, hotels) have no standard API to check identity.
- No audit trail exists for data access — citizens cannot see who accessed their information.
- Manual KYC processes take days; NagarikID reduces this to under 2 seconds.

Core capabilities

- Citizen authentication via National ID number + bcrypt password + biometric face-match simulation.
- Verifier organization login by Company PAN with business-type-scoped JWT.
- Policy engine: maps business_type → JSONB allowlist of fields. Banks see addresses. Pharmacies see blood type only. Nothing more.
- Append-only audit trail — every verification decision stored with IP hash, fields accessed, result, and timestamp.
- Suspicious activity detection — brute force, rate limit breach, replay attacks auto-flagged.
- OCR document scan simulation — extracts citizen data from uploaded citizenship card images.

System actors

Actor	Role
Citizen	Logs in with NID + biometric, reviews audit trail, manages consent, revokes org access
Verifier (Org)	Bank, pharmacy, hotel — authenticated by PAN, receives only policy-allowed fields
Admin	Government operator — views all citizens, threats, audit trail, manages policies
System (API)	Express middleware chain — enforces policy, logs every decision, blocks threats

SECTION 2

Master AI Generation Prompt

Copy the prompt below verbatim into any capable AI code generation tool (Claude, GPT-4o, Cursor, v0.dev, etc.) to scaffold the full NagarikID PERN platform. The prompt is structured in layers — system context, technical constraints, feature requirements, and design spec — so the AI has everything it needs to produce working code without hallucinating dependencies or architecture.

TIP — For best results: paste this into Claude Sonnet 4.5 or GPT-4o with a 200K+ context window. Run in Cursor or a VS Code AI extension for direct file generation. For the frontend only, use v0.dev with the design subsection of this prompt.

Full System Prompt

You are a senior full-stack engineer. Build the complete NagarikID digital identity platform — a production-grade PERN stack system (PostgreSQL, Express.js, React via Next.js, Node.js) for Nepal's national KYC and identity verification infrastructure.

SYSTEM CONTEXT

NagarikID connects Nepal's National ID (NID) and Nagarik App into a self-maintained digital identity system. Citizens control data sharing. Organizations get only what policy allows. Every access decision is permanently logged.

Three portals:

1. citizen-app — Next.js 14, citizen login + biometric + audit trail
2. verifier-scanner — Next.js 14, org login + policy-scoped verification
3. backend — Express.js API, JWT auth, policy engine, audit logger

One database:

PostgreSQL on Supabase (or Neon). Schema has 6 tables.

TECHNICAL CONSTRAINTS — HARD REQUIREMENTS

Node.js version: 20 LTS

Package manager: npm

TypeScript: enabled in all projects

ORM: pg (raw driver) — NOT Prisma, NOT Sequelize

Auth: JWT (jsonwebtoken) with RS256 signing

Password: bcrypt with cost factor 12

Validation: Zod on all API inputs

Rate limiting: express-rate-limit with memory store (Redis optional)

CORS: strict whitelist from env vars only
 Frontend state: Zustand for auth state, React Query for server state
 Styling: Tailwind CSS 3 + shadcn/ui components
 Animations: Framer Motion 11 – scroll-driven reveals, page transitions
 Font: Nunito (headings) + Open Sans (body) via next/font/google
 No ORMs. No GraphQL. REST only.

DATABASE SCHEMA – implement exactly

Run in this order:

```
-- 1. users (auth root)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  national_id  VARCHAR(20) UNIQUE,
  password_hash TEXT NOT NULL,
  role        VARCHAR(20) CHECK (role IN ('citizen','verifier','admin')),
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  last_login  TIMESTAMPTZ
);

-- 2. citizen_profiles
CREATE TABLE citizen_profiles (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  full_name   VARCHAR(120) NOT NULL,
  dob        DATE NOT NULL,
  gender      VARCHAR(20),
  address     TEXT,
  phone      VARCHAR(20),
  blood_group VARCHAR(4),
  photo_url   TEXT,
  biometric_score FLOAT DEFAULT 0,
  status      VARCHAR(20) DEFAULT 'pending'
  CHECK (status IN ('pending','verified','suspended'))
);

-- 3. verifier_organizations
CREATE TABLE verifier_organizations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  org_name    VARCHAR(200) NOT NULL,
  company_pan VARCHAR(20) UNIQUE NOT NULL,
  business_type VARCHAR(30) CHECK (business_type IN
('bank','pharmacy','age_verification')),
  is_active   BOOLEAN DEFAULT true,
  license_expires DATE
);

-- 4. permission_policies
CREATE TABLE permission_policies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_type VARCHAR(30) UNIQUE NOT NULL,
  allowed_fields JSONB NOT NULL,
```

```

    policy_version INT DEFAULT 1,
    updated_at     TIMESTAMPTZ DEFAULT NOW()
);
-- Seed the policy table:
INSERT INTO permission_policies (business_type, allowed_fields) VALUES
    ('bank',          '['full_name","dob","address","phone"]'),
    ('pharmacy',      '['full_name","dob","blood_group"]'),
    ('age_verification', '['full_name","dob"]');

-- 5. audit_trail (append-only - NEVER grant UPDATE/DELETE)
CREATE TABLE audit_trail (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    citizen_id        UUID REFERENCES citizen_profiles(id),
    verifier_id        UUID REFERENCES verifier_organizations(id),
    fields_accessed    JSONB NOT NULL,
    purpose            TEXT,
    result             VARCHAR(16) CHECK (result IN ('APPROVED','BLOCKED')),
    ip_hash            VARCHAR(64),
    created_at         TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_audit_citizen  ON audit_trail(citizen_id, created_at DESC);
CREATE INDEX idx_audit_verifier ON audit_trail(verifier_id);

-- 6. suspicious_activity
CREATE TABLE suspicious_activity (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id            UUID REFERENCES users(id),
    type              VARCHAR(30) CHECK (type IN
('brute_force','replay','rate_limit','anomaly')),
    ip_address         INET,
    attempt_count      INT DEFAULT 1,
    status             VARCHAR(20) DEFAULT 'flagged'
    CHECK (status IN ('flagged','blocked','resolved')),
    flagged_at         TIMESTAMPTZ DEFAULT NOW()
);

```

Backend Prompt (Express API)

Build the Express.js backend with this exact structure:

```

backend/
  src/
    app.ts           - Express app factory (no listen here)
    server.ts        - process entry, calls app.listen
  db/
    pool.ts          - pg Pool, sslmode=require, connection test on boot
    queries/         - one file per table, typed query functions
  middleware/
    rateLimiter.ts   - express-rate-limit, 120/min global, 10/min auth
    cors.ts          - CORS with env var whitelist
    auth.ts          - JWT verify middleware, attaches req.user

```

```

    requireRole.ts      - role guard factory: requireRole('admin')
    validate.ts         - Zod schema validation middleware factory
    auditLogger.ts      - async insert to audit_trail, never throws
    threatDetector.ts   - threshold check, inserts suspicious_activity
    errorHandler.ts     - central 4-arg Express error handler
    routes/
      auth.ts           - /api/v1/auth/* (citizen login, verifier login,
refresh, logout)
      citizens.ts       - /api/v1/citizens/me, /api/v1/citizens/me/audit-trail
      verifications.ts  - /api/v1/verifications/execute (policy engine lives
here)
      admin.ts         - /api/v1/admin/* (all require admin role)
      health.ts        - /api/v1/health (db ping + version + uptime)
    services/
      policyEngine.ts   - queries permission_policies, projects only allowed
fields
      biometric.ts     - simulated face-match scoring (returns 94-99.9 score)
      ocr.ts           - simulated OCR extraction from base64 image
    types/
      express.d.ts     - augment req with req.user typed
      index.ts         - shared types (CitizenProfile, VerifierOrg, etc.)

```

Middleware chain (app.ts, this exact order):

1. helmet() - security headers
 2. rateLimiter - global rate limit
 3. cors - origin whitelist
 4. express.json() - body parser
 5. morgan('combined') - request logging in production
- Routes mounted here
6. errorHandler - must be last

Auth flow:

```

POST /api/v1/auth/citizen/login
Body: { national_id, password }
1. Query users WHERE national_id = $1
2. bcrypt.compare(password, hash)
3. Run biometric service (simulated - returns score)
4. Sign access token (15m, RS256) + refresh token (7d)
5. Set refresh in httpOnly cookie
6. Return: { access_token, citizen_profile, biometric_score }

```

```

POST /api/v1/verifications/execute
Header: Authorization: Bearer <verifier_access_token>
Body: { national_id, purpose }
1. Verify JWT, extract verifier org from token
2. Lookup citizen by national_id
3. Query permission_policies WHERE business_type = verifier.business_type
4. Project citizen_profile fields: only keep keys in allowed_fields JSONB array
5. Insert into audit_trail (async, do not await in response path)
6. Run threatDetector in background
7. Return: { result: 'APPROVED', data: { ...filtered fields } }

```

Error response format (always):

```
{ success: false, error: string, code?: string }
```

```
Success response format (always):
{ success: true, data: any, meta?: { page, total, ... } }
```

Frontend Prompt (citizen-app)

Build citizen-app as Next.js 14 with App Router.

```
citizen-app/
  app/
    layout.tsx          - root layout, Nunito+OpenSans fonts, Toaster
    page.tsx            - redirects to /login if no session
    (auth) /
      login/page.tsx    - NID + password form, then biometric step
      register/page.tsx - OCR scan tab + manual registration form
    (dashboard) /
      layout.tsx        - sidebar nav + session guard
      overview/page.tsx - profile card + stats
      audit-trail/page.tsx - paginated access history
      consent/page.tsx  - per-org consent toggles with revoke
      privacy/page.tsx  - field visibility map
    components /
      BiometricScan.tsx - animated face scan with landmark dots
      OCRScanner.tsx    - drag-drop upload + extraction animation
      AuditRow.tsx      - single audit event with risk badge
      ConsentCard.tsx   - org access toggle
      NavSidebar.tsx    - responsive sidebar
      StatusBadge.tsx   - VERIFIED / PENDING / BLOCKED chip
    lib /
      api.ts            - typed fetch wrapper, attaches Authorization header
      auth.ts           - Zustand store: session, login(), logout()
      hooks/useAuditTrail.ts - React Query hook for audit trail
```

Design rules (do not deviate):

```
Background: #FAFAFA (page), #FFFFFF (cards)
Text:       #111111 (headings), #374151 (body), #6B7280 (muted)
Accent:     #4F46E5 (indigo) for interactive elements
Success:    #059669 green
Error:      #DC2626 red
Border:     1px solid #E5E7EB
Radius:     8px cards, 100px pills/badges
Font:       Nunito 700/800 for headings, Open Sans 300/400/500 for body
Motion:     Framer Motion - viewport-triggered fadeUp (y:24, opacity:0→1)
            stagger delay 0.08s per child
            spring easing: { type:'spring', stiffness:100, damping:20 }
No shadows except: 0 2px 8px rgba(0,0,0,0.04) on cards
No gradients except hero section background
```

Frontend Prompt (verifier-scanner)

Build verifier-scanner as a second Next.js 14 project (separate Vercel deploy).

```
verifier-scanner/  
  app/  
    login/page.tsx      - PAN + password login  
    dashboard/  
      page.tsx          - scan form + live result panel  
      history/page.tsx  - session verification log  
  components/  
    ScanForm.tsx        - NID input + purpose selector + execute button  
    PolicyBadge.tsx     - shows allowed fields for this org type  
    ResultPanel.tsx     - APPROVED/BLOCKED status + filtered field display  
    FieldRow.tsx        - single field name + value with redaction state
```

The result panel must show:

1. APPROVED / BLOCKED badge (green/red)
2. Only fields returned by the API (policy-filtered server-side)
3. A note: 'X fields redacted by policy' where X = total fields - returned fields
4. Biometric confidence score
5. Timestamp of verification
6. Audit trail ID for this verification

SECTION 3

Database Architecture

PostgreSQL is the single source of truth. All six tables are intentional — no table is optional. The schema is designed for append-heavy audit workloads (audit_trail), low-cardinality policy lookups (permission_policies), and fast citizen lookups by national_id.

Table Relationship Map

Table	Relation	Key fields
users	Root auth table	national_id UNIQUE, role ENUM, password_hash
citizen_profiles	FK → users	PII: name, dob, address, phone, blood_group, photo_url
verifier_organizations	FK → users	org_name, company_pan UNIQUE, business_type
permission_policies	Lookup table	business_type → allowed_fields JSONB array
audit_trail	FK → citizen + verifier	Append-only, indexed by citizen_id + created_at
suspicious_activity	FK → users	type, ip_address INET, attempt_count, status

Critical DB rules

- Never grant UPDATE or DELETE on audit_trail to the application user. Use a separate read-only user for analytics.
- Run all SQL scripts in order: schema.sql → migrate_drugs.sql → migrate_photos.sql → migrate_suspicious.sql.
- Enable sslmode=require on all connections. Supabase enforces this by default.
- Create indexes after inserting seed data, not before — Postgres builds them faster on existing data.
- Use gen_random_uuid() not uuid_generate_v4() — the former is available without extension.
- The allowed_fields column in permission_policies is JSONB not TEXT[]. Query with: allowed_fields @> '{"full_name"}':jsonb

Recommended indexes

```
-- citizen lookup (login + verification hot path)
CREATE INDEX idx_users_national_id ON users(national_id);
```

```
-- audit trail queries (citizen views their history)
CREATE INDEX idx_audit_citizen      ON audit_trail(citizen_id, created_at DESC);
CREATE INDEX idx_audit_verifier    ON audit_trail(verifier_id);

-- threat detection (admin dashboard)
CREATE INDEX idx_suspicious_ip      ON suspicious_activity(ip_address, flagged_at
DESC);
CREATE INDEX idx_suspicious_user    ON suspicious_activity(user_id);

-- policy lookup (every verification request hits this)
-- Already fast via business_type UNIQUE — no extra index needed.
```

SECTION 4

Backend API Specification

Endpoint Reference

Method	Path	Auth	Description
POST	/api/v1/auth/citizen/login	Public	NID + password → JWT + biometric score
POST	/api/v1/auth/verifier/login	Public	PAN + password → scoped JWT
POST	/api/v1/auth/refresh	Refresh token cookie	Rotate access token
POST	/api/v1/auth/logout	Any JWT	Clear cookie, invalidate session
GET	/api/v1/citizens/me	Citizen JWT	Own profile — no unrequested fields
GET	/api/v1/citizens/me/audit-trail	Citizen JWT	Paginated access history
POST	/api/v1/verifications/execute	Verifier JWT	Policy engine → filtered citizen data
GET	/api/v1/admin/citizens	Admin JWT	All citizens, paginated
GET	/api/v1/admin/suspicious	Admin JWT	Threat feed with filters
GET	/api/v1/health	None	DB ping + uptime + version

Middleware Chain

Every request passes through this chain in order. Order is non-negotiable — changing it breaks auth or leaks data.

Step	Middleware	Detail
1	helmet()	Sets 11 security headers: CSP, HSTS, X-Frame-Options, Referrer-Policy, etc.
2	rateLimiter	120 req/min global · 10/min on /auth routes · memory store
3	cors()	Strict whitelist: only CORS_ORIGIN_CITIZEN and CORS_ORIGIN_VERIFIER env vars
4	express.json()	Parse JSON bodies, limit: 10mb
5	morgan()	Request logging — 'combined' in prod, 'dev' in development
—	Route handlers	auth.ts, citizens.ts, verifications.ts, admin.ts, health.ts

6	errorHandler	4-arg Express handler — strips stack in prod, structured JSON errors
---	--------------	--

Policy Engine logic

```
// services/policyEngine.ts
export async function applyPolicy(
  citizenProfile: CitizenProfile,
  businessType: string,
  db: Pool
): Promise<Partial<CitizenProfile>> {

  // 1. Fetch allowed fields for this org type
  const { rows } = await db.query(
    'SELECT allowed_fields FROM permission_policies WHERE business_type = $1',
    [businessType]
  );
  if (!rows.length) throw new AppError('No policy found', 403);

  const allowedFields: string[] = rows[0].allowed_fields;

  // 2. Project — only keep allowed keys
  const filtered: Record<string, unknown> = {};
  for (const field of allowedFields) {
    if (field in citizenProfile) {
      filtered[field] = citizenProfile[field as keyof CitizenProfile];
    }
  }

  return filtered as Partial<CitizenProfile>;
}
```

SECTION 5

Frontend Design System

Both Next.js apps share a common design language. The system uses Tailwind CSS 3 with a custom config, shadcn/ui primitives, and Framer Motion for scroll-driven animations. No component library beyond shadcn/ui. No inline styles.

Color tokens

Token	Value	Usage
--color-bg	#FAFAFA	Page background
--color-surface	#FFFFFF	Card surfaces
--color-border	#E5E7EB	All borders (1px solid)
--color-text-primary	#111111	Headings, high-emphasis text
--color-text-body	#374151	Body copy
--color-text-muted	#6B7280	Supporting text, labels
--color-accent	#4F46E5	Indigo — buttons, links, focus rings
--color-success	#059669	Verified badges, APPROVED status
--color-error	#DC2626	BLOCKED, threat alerts
--color-warning	#D97706	Pending, caution states

Typography

Font	Usage	Spec
Nunito 800	Page H1	clamp(36px, 5vw, 64px) · tracking -0.03em
Nunito 700	Section H2	clamp(24px, 3.5vw, 42px) · tracking -0.02em
Nunito 600	Card titles H3	18px · tracking -0.01em
Open Sans 400	Body text	15px · line-height 1.75 · color text-body
Open Sans 300	Supporting text	13–14px · color text-muted
Open Sans 600	Labels, badges, CTAs	11–12px · tracking 0.04–0.12em · UPPERCASE
Courier New / monospace	Code, IDs, tokens	13px · color text-body

Animation rules

- All page-entry animations use Framer Motion viewport trigger — animate when element enters viewport, not on mount.
- FadeUp: opacity 0→1, y 24→0. Duration 0.65s. Spring: stiffness 100, damping 20.
- Stagger children: 0.08s delay between each child in a list or grid.
- Hover on cards: translateY(-4px), box-shadow increase. Duration 0.25s ease.
- Page transitions: fade opacity 0→1 over 0.35s on route change using AnimatePresence.
- Biometric scan animation: IntersectionObserver, CSS keyframes for scan beam + dot reveal, no JS animation loop.
- Reduced motion: wrap all animations with useReducedMotion() hook. If true, skip transforms, keep opacity only.

Component hierarchy

```
// Shared design tokens in tailwind.config.ts
theme: {
  extend: {
    fontFamily: {
      heading: ['Nunito', 'sans-serif'],
      body: ['Open Sans', 'sans-serif'],
    },
    colors: {
      accent: { DEFAULT: '#4F46E5', light: '#EEF2FF' },
      success: { DEFAULT: '#059669', light: '#ECFDF5' },
      danger: { DEFAULT: '#DC2626', light: '#FEF2F2' },
    },
    borderRadius: {
      card: '12px',
      pill: '100px',
    },
  },
}
```

SECTION 6

Authentication & Security

JWT token architecture

Token type	Specification
Access token	RS256 signed · 15 min TTL · Payload: { sub, role, org_id }
Refresh token	RS256 signed · 7 day TTL · httpOnly cookie · Secure · SameSite=Strict
Admin token	Same as access · role='admin' · No refresh — re-login required every 15 min

IMPORTANT — Key generation: Generate RS256 key pair with: openssl genrsa -out private.pem 2048 && openssl rsa -in private.pem -pubout -out public.pem. Store private key as JWT_PRIVATE_KEY env var (base64 encoded). Never commit keys to git.

Security checklist

- bcrypt cost factor: 12 (never lower — 10 is too fast for 2026 hardware)
- Refresh tokens stored in httpOnly, Secure, SameSite=Strict cookies only — never in localStorage
- All inputs validated with Zod before touching the database — no raw SQL interpolation
- Rate limiting: 10 requests per 15 minutes on /auth routes — auto-inserts to suspicious_activity on breach
- CORS: hardcoded allowlist from env vars — never wildcard (*) in production
- Helmet: enables Content-Security-Policy, HSTS, X-Frame-Options, referrer-policy
- Database connection: sslmode=require — reject connection if SSL unavailable
- Secrets rotation: every 90 days — use CI/CD secret injection, never .env in repo
- audit_trail: grant only INSERT + SELECT to app DB user — never UPDATE or DELETE
- IP hashing in audit_trail: SHA-256 of IP address — stores hash, not raw IP (privacy)

Threat detection thresholds

Threat type	Trigger condition	Response
Brute force	5 failed logins on same national_id in 15 min	Insert suspicious_activity, lock account 30 min
Rate limit	120 req/min from same IP global	429 response, log to suspicious_activity

Replay attack	Same JWT used after logout (token blacklist check)	401, flag to suspicious_activity
Anomaly	Verifier org > 500 requests/hour	Alert admin webhook, soft throttle to 50/hour

SECTION 7

AI & Biometric Integration

In the simulated production build, biometric face-matching and OCR document scanning are implemented as deterministic service functions. In a real deployment, these would call the government's biometric verification API or a licensed OCR provider.

Biometric service (simulated)

```
// services/biometric.ts
// In production: replace with call to govt biometric API or face-api.js

export interface BiometricResult {
  score: number;           // 0-100 confidence
  passed: boolean;         // true if score >= THRESHOLD
  landmarks: number;       // face landmarks detected (68 standard)
  liveness: boolean;       // anti-spoofing check passed
}

const THRESHOLD = 90;

export async function verifyFace(
  citizenId: string,
  capturedImageBase64: string
): Promise<BiometricResult> {
  // Simulate processing delay (200-600ms like a real API)
  await new Promise(r => setTimeout(r, 200 + Math.random() * 400));

  // Deterministic score based on citizenId hash (consistent per user)
  const baseScore = 92 + (parseInt(citizenId.slice(-2), 10) % 7);
  const score = Math.min(99.9, baseScore + Math.random() * 1.5);

  return {
    score: parseFloat(score.toFixed(1)),
    passed: score >= THRESHOLD,
    landmarks: 68,
    liveness: true,
  };
}
```

OCR service (simulated)

```
// services/ocr.ts
// In production: replace with Tesseract.js, Google Vision, or AWS Textract

export interface OCRResult {
```

```
confidence: number;      // 0-100
extracted: {
  full_name?: string;
  dob?: string;
  national_id?: string;
  address?: string;
};
warnings: string[];
}

export async function extractFromIDCard(
  imageBase64: string,
  mimeType: string
): Promise<OCRResult> {
  // Simulate OCR processing (500ms-1.5s)
  await new Promise(r => setTimeout(r, 500 + Math.random() * 1000));

  // Return simulated extraction
  return {
    confidence: 97 + Math.random() * 2.9,
    extracted: {
      full_name: 'Extracted from document',
      dob: '1990-01-01',
      national_id: '0000000000',
      address: 'Kathmandu, Nepal',
    },
    warnings: [],
  };
}
```

Real deployment path

- Face matching: Integrate face-api.js (client-side TensorFlow.js) for in-browser biometric — no image ever leaves the device.
- Government NID API: Replace biometric.ts with a call to Nepal's NID verification endpoint (requires government API key and MOU).
- OCR: Tesseract.js for client-side OCR (Citizen app) or AWS Textract for server-side (higher accuracy, add cost).
- Liveness detection: Add MediaPipe Face Mesh for blink detection + head turn — prevents photo spoofing.

SECTION 8

Deployment Runbook

Step 1 — Provision Database

- Create PostgreSQL instance on Supabase (free tier supports this project).
- Enable SSL in Supabase dashboard → Settings → Database → SSL.
- Open SQL editor. Run scripts in this exact order:
 - database/schema.sql
 - database/migrate_drugs.sql
 - database/migrate_photos.sql
 - database/migrate_suspicious.sql
- Copy the connection string from: Settings → Database → Connection string → URI mode.
- Confirm sslmode=require is appended to the URI.

Step 2 — Deploy Backend (Render)

- Create a new Web Service on Render. Connect your GitHub repo.
- Set: Root Directory = backend · Build Command = npm ci · Start Command = npm start
- Select: Node 20 · Region: Southeast Asia (Singapore) for lowest Nepal latency.
- Add all 13 environment variables (see Section 9).
- Deploy. Note the URL: https://<name>.onrender.com
- Validate: curl https://<name>.onrender.com/api/v1/health

Step 3 — Deploy citizen-app (Vercel)

- Import repo into Vercel. Set Root Directory = citizen-app · Framework Preset = Next.js.
- Add environment variables: NEXT_PUBLIC_API_BASE = https://<backend-url>.onrender.com
- Deploy. Note URL: https://<name>.vercel.app

Step 4 — Deploy verifier-scanner (Vercel)

- Create a second Vercel project from the same repo.
- Root Directory = verifier-scanner. Same API base URL env var.
- Deploy. Note URL: https://<verifier-name>.vercel.app

Step 5 — CORS alignment

- Go to Render backend env vars. Update:
 - CORS_ORIGIN_CITIZEN = https://<citizen-name>.vercel.app
 - CORS_ORIGIN_VERIFIER = https://<verifier-name>.vercel.app
- Trigger a manual redeploy of the backend.

Step 6 — Smoke test

```
# Health check
curl https://<api>/api/v1/health

# Citizen login
curl -X POST https://<api>/api/v1/auth/citizen/login \
  -H 'Content-Type: application/json' \
  -d '{"national_id":"9876543210","password":"Nepal@123"}'

# Verifier login
curl -X POST https://<api>/api/v1/auth/verifier/login \
  -H 'Content-Type: application/json' \
  -d '{"company_pan":"PAN001","password":"Bank@2024"}'

# After verifier login – execute verification
curl -X POST https://<api>/api/v1/verifications/execute \
  -H 'Authorization: Bearer <verifier_access_token>' \
  -H 'Content-Type: application/json' \
  -d '{"national_id":"9876543210","purpose":"KYC Verification"}'

# Confirm audit trail was written
# Login as citizen, then:
curl https://<api>/api/v1/citizens/me/audit-trail \
  -H 'Authorization: Bearer <citizen_access_token>'
```

SECTION 9

Environment Variables

All 13 variables are required. The backend will refuse to start if DATABASE_URL or JWT_SECRET are missing. Use the platform's secret management UI — never commit .env files to git.

Variable	Example value	Notes
DATABASE_URL	postgresql://user:pass@host:5432/nagarikid?sslmode=require	Supabase/Neon/RDS URI with SSL enforced
JWT_SECRET	<256-bit random string>	openssl rand -hex 32
JWT_EXPIRES_IN	15m	15 minutes — short-lived access token
REFRESH_TOKEN_SECRET	<256-bit random string, different>	Must differ from JWT_SECRET
REFRESH_TOKEN_EXPIRES_IN	7d	7 days — stored in httpOnly cookie
HMAC_SECRET	<256-bit random string, different>	For IP hashing in audit_trail
CORS_ORIGIN_CITIZEN	https://citizen.nagarikid.gov.np	Exact origin — no trailing slash
CORS_ORIGIN_VERIFIER	https://verifier.nagarikid.gov.np	Exact origin — no trailing slash
BCRYPT_ROUNDS	12	Minimum 12 — never lower
RATE_LIMIT_WINDOW_MS	60000	1 minute window in milliseconds
RATE_LIMIT_MAX	120	Max requests per window per IP
PORT	10000	Render uses 10000 by default
NODE_ENV	production	Disables stack traces in error responses

Generate all three secrets at once: openssl rand -hex 32 && openssl rand -hex 32 && openssl rand -hex 32. Each output is a different secret. Copy them into your deployment platform's environment variable manager.

SECTION 10

Implementation Checklist

Use this checklist when scaffolding the system manually or reviewing AI-generated output. Every item marked is a hard requirement — not a recommendation.

Database

- All 6 tables created in correct order with exact column types
- permission_policies seeded with bank, pharmacy, age_verification policies
- All 4 indexes created after seeding
- App DB user granted INSERT+SELECT only on audit_trail
- Connection string tested with sslmode=require

Backend

- package.json: Node 20 engine field set
- TypeScript strict mode enabled in tsconfig.json
- Zod validation on every route input
- JWT RS256 keys generated and base64-encoded in env vars
- Middleware chain order verified: helmet → rate → cors → json → morgan → routes → error
- Rate limiter: 10/min on /auth routes specifically
- Refresh token in httpOnly cookie with Secure and SameSite=Strict
- auditLogger runs async — never awaited in response path
- errorHandler strips stack trace when NODE_ENV=production
- Health endpoint returns DB latency in response
- All routes return consistent { success, data } or { success, error } shape

Frontend — citizen-app

- Fonts loaded via next/font/google — no external CDN
- Auth state in Zustand store — not Context or Redux
- Server state via React Query — not useEffect fetch
- All API calls go through lib/api.ts typed wrapper
- BiometricScan component: scan beam CSS animation, 68 landmark dots
- Framer Motion: viewport-triggered fadeUp on all major sections
- Reduced motion: useReducedMotion() hook wrapping all transforms
- Consent manager: per-org toggle with instant optimistic UI update
- Audit trail: paginated table with risk badge coloring

Frontend — verifier-scanner

- Separate Vercel project — not a subdirectory of citizen-app
- Policy badge shows allowed fields before verification is executed
- Result panel shows count of redacted fields
- No citizen PII cached on frontend — only display what API returns

Security

- No plaintext passwords anywhere in logs, responses, or comments
- IP hashing confirmed: `audit_trail.ip_hash` = SHA-256, not raw IP
- CORS tested: requests from unknown origins return 403
- Brute force: 5 failed logins trigger `suspicious_activity` insert
- Token expiry tested: expired access token returns 401
- Secrets differ: `JWT_SECRET` ≠ `REFRESH_TOKEN_SECRET` ≠ `HMAC_SECRET`

Deployment

- Smoke tests pass: health, citizen login, verifier login, verification execute
- `audit_trail` has new rows after smoke test verifications
- CORS origins updated to real Vercel URLs (not localhost)
- Backend redeployed after CORS update
- CI/CD: main branch push triggers automatic redeploy on all three services

NagarikID · Digital Identity Platform · Master Specification v2.0
Built for Nepal. Designed for scale. Open for audit.